

Interim Report

Templating in Standard ML

Ja Ying Lew

Supervisor: Gergely Buday

Department of Biological, Chemical and Materials Engineering

University of Sheffield

December 6, 2025

Contents

1	Background to Project	3
1.1	Mustache Template Example: Student Profile Generation	3
2	Standard ML and Mustache	6
3	Literature Review	8
3.1	Theoretical Framework	8
3.2	Template System Design	8
3.3	Alternative Templating Systems	9
3.4	Functional Languages for Template Processing	10
3.5	Takeaway	11
4	Aims and Objectives	12
4.1	Proposal	12
4.2	Research Questions	12
4.3	Technical Approach	13
4.4	Learning Objectives	13
5	Project Management	14
5.1	Project Overview and Timeline	14
5.2	Key Milestones	14
5.3	Planned Work Schedule (October - December 2025)	14
5.3.1	Weeks 1-3 (Early October): Exploration and Topic Selection . . .	14
5.3.2	Weeks 4-6 (Mid-Late October): Topic Refinement	15
5.3.3	Weeks 7-9 (November): Literature Review	15
5.3.4	Weeks 10-11 (Early December): Interim Report Finalisation . . .	15
5.4	Forward Plan (January - May 2026)	16
5.4.1	Weeks 12-13 (Christmas Break): SML Implementation Foundations	16
5.4.2	Weeks 14-17 (Late January - February): Pattern Matching Deep Dive	16
5.4.3	Weeks 18-19 (Late February - Early March): Mustache Implemen- tation	16

5.4.4	Weeks 20-22 (March): Testing and Refinement	17
5.4.5	Weeks 23-26 (April - Early May): Dissertation Writing	17
5.4.6	Week 27 (Mid-May): Final Submission	17
5.5	Deliverables and Success Criteria	17
5.6	Dependencies and Critical Path	17
5.6.1	Sequential Dependencies	18
5.7	Risk Management and Mitigation	18
6	Current Status of Work	20
6.1	Overview	20
6.2	Progress by Phase	20
6.2.1	Weeks 1-3 (Early October): Foundation	20
6.2.2	Weeks 4-6 (Mid-Late October): Topic Refinement	20
6.2.3	Weeks 7-9 (November): Literature Deep Dive	20
6.2.4	Weeks 10-11 (Early December): Finalization	21
6.3	Key Accomplishments	21
6.4	Self-Review	21
A	Project Timeline	23

1. Background to Project

Template systems are generally used for the development of web-based systems, which are generated dynamically through a programming language while the static part of the page is manually written HTML. Most template systems separate markup and tags from application logic, particularly Mustache highlights this through its "logic-less" design by prohibiting embedding traditional conditional statements or loops within templates [1]. The strict separation of all logic residing in the view model and templates being able to format specifications has led to Mustache's widespread adoption across numerous programming languages. [2] Despite having a broad implementation and being able to provide readable output, loopholes arise when they require preprocessing/ massaging of the data to produce a 'logical' output [3–5]. Exploiting standard ML's strong static type system, strong parsing and pattern matching abilities, these characteristics may guarantee template correctness and type-safe data processing[6, 7].

1.1. Mustache Template Example: Student Profile Generation

All logic must be pre-computed in the data structure, and the template merely renders what is provided. This separation ensures a clear boundary between presentation and business logic. Consequently, there is no string concatenation, arithmetic, or function calls are possible within the template and all formatting decisions (eg, commas or comments) require pre-processing.

Input Data

The JSON data structure contains all pre-computed values, including boolean flags that determine which sections to render, sometimes the tags may miss certain pre-computed values. This necessitates a good `.json`:

Listing 1.1: Input data (data.json)

```
1 {
2   "vars": {
3     "people": [
```

```

4      {
5          "name": "JohnSmith",
6          "email": "john@example.com",
7          "degree": "BSc Computer Science",
8          "isComputerScience": true,
9          "skills": [
10             {"skill": "Python"},
11             {"skill": "JavaScript"},
12             {"skill": "SQL", "last": true}
13         ]
14     }
15 ]
16 }
17 }

```

Template Structure

Listing 1.2: Mustache template (example.mustache)

```

1  {{#vars}}                // Section: only renders if vars exists
2  {{#people}}              // Iteration: renders once per person
3  {{#isComputerScience}}   // Conditional: renders if true
4  // Computer Science Student // No if/else logic - just presence check
5  public class {{name}}ComputerScienceProfile
6  {
7      public string Name = "{{name}}";    // Simple substitution
8      public string Email = "{{email}}";  // No string manipulation
9      public string Degree = "{{degree}}"; // No computation
10
11      public bool IsComputerScience = true;
12      public string Specialization = "Software Development";
13  {{/isComputerScience}}
14  {{^isComputerScience}}    // Inverted: renders if false
15  // Non-Computer Science Student // Still no logic - just absence check
16  public class {{name}}NonComputerScienceProfile
17  {
18      public string Name = "{{name}}";
19      public string Email = "{{email}}";
20      public string Degree = "{{degree}}";
21  {{/isComputerScience}}
22

```

```

23     public string[] Skills = { {{#skills}}{{skill}}{{^last}}, {{/last}}{{/
        skills}} };
24 }                                     // Array iteration with separator
25
26 {{/people}}
27 {{/vars}}

```

Generated Output

The rendered output shows how the template collects the desired information based on the provided data:

Listing 1.3: Generated C# output

```

1  // Computer Science Student
2  public class JohnSmithComputerScienceProfile
3  {
4      public string Name = "JohnSmith";
5      public string Email = "john@example.com";
6      public string Degree = "BSc Computer Science";
7
8      public bool IsComputerScience = true;
9      public string Specialization = "Software Development";
10
11     public string[] Skills = { "Python", "JavaScript", "SQL" };
12 }

```

2. Standard ML and Mustache

Mustache addressed the challenge of generating dynamic web content while maintaining strict separation between presentation and application logic in Ruby on Rails applications by enforcing its "logic-less" paradigm [5]. This distinguishable characteristic ensures consistent behaviour across all implementations [5]. Templates use double curly braces (`{{variable}}`) for variable substitution, section tags (`{#section}` ... `{/section}`) for iteration and conditionals, inverted sections (`{^section}`) for negative conditions, and partials (`{partial}`) for template composition. This design makes templates portable across programming environments and readable by non-programmers, while HTML escaping is applied by default to prevent cross-site scripting attacks.

However, this is disputed as it still has control statements (ie logic) implicitly as it uses 'section tags processing lists and anonymous functions (lambdas)' [5].

Mustache has achieved implementations available in over 40 programming languages including ActionScript, C++, Clojure, Java, JavaScript, Python, Ruby, Rust, Haskell, and OCaml [2]. Current applications span mobile and web development, configuration file generation, source code generation, build automation tools, and API client generation through systems like OpenAPI Generator [5, 8].

However, the design necessitates extensive pre (ie exceptional clear .json) or post (ie. to manually check whether the view is correct) processing: as templates grow in size and complexity, they become increasingly difficult to manage without the flexibility of conditional logic [3]. Every variation to the view requires updating both the view model and template, increasing maintenance burden.

Standard ML's design characteristics make it particularly suitable for template processing tasks. Its Hindley-Milner type system provides automatic type inference while ensuring type safety through formal guarantees that well-typed programs cannot cause runtime type errors [9]. Standard ML's algebraic datatypes, combined with exhaustive pattern matching, naturally represent abstract syntax trees; as VandenBerghe notes, ML's type constructors are "wonderful for describing ASTs," with pattern matching making code "incredibly readable" for functions on complex structures [7].

These characteristics directly address Mustache's preprocessing requirements demonstrated in Section 1.1. Algebraic types can represent both template syntax trees and JSON data structures, while pattern matching enables systematic handling of nested sec-

tions and conditional flags. Crucially, exhaustive pattern matching ensures all cases are covered. The compiler verifies that every required preprocessing transformation (boolean flags, separator markers, format variations) is handled, catching omissions at compile-time rather than runtime.

However, Standard ML’s ecosystem limitations present practical challenges. The language has significantly fewer third-party libraries than mainstream alternatives [7], requiring custom implementation of facilities like JSON parsing that would be readily available in Python or JavaScript. This tradeoff of accepting implementation challenge for compile-time correctness guarantees, frames the project’s core question: can Standard ML’s type safety justify the additional development effort in template preprocessing contexts?

While Mustache’s preprocessing burden is well-documented as a usability complaint [3, 4], it has not been formalized or addressed through type-safe verification. Standard ML’s type system could verify preprocessing completeness at compile-time (catching missing flags/markers before runtime) or validate postprocessing correctness (ensuring rendered output meets specifications). No existing work explores this intersection of formal methods and template preprocessing. The literature review evaluates whether alternative template systems avoid this burden entirely, or whether all logic-restricted approaches face similar challenges requiring systematic solutions.

3. Literature Review

3.1. Theoretical Framework

This review evaluates template systems through Parr’s (2004) formal framework for model-view separation, which defines five ”entanglement” violations: conditional logic based on view structure, method/function calls from templates, value computation within templates, model scope manipulation, and recursive rendering [10]. Parr shows that unrestricted templates inevitably violate separation when developers embed business logic, as templates end up producing logic, with template computational power classified from weakest (substitution-only) to strongest (Turing-complete). Pisu et al.’s (2024) security analysis confirms these theoretical predictions: examining 34 template engines, they found 71% of server-side template injection (SSTI) vulnerabilities occur in engines permitting conditional logic, with Jinja2’s sandbox suffering documented bypasses (CVE-2024-22195, CVE-2024-27516) [11]. This establishes a fundamental tension: strict separation (Mustache) enforces security but imposes preprocessing burden; unrestricted power (Jinja2, EJS) reduces preprocessing but enables separation violations and security exploits.

3.2. Template System Design

Template systems separate presentation markup from application code through ’separation of concerns’: business ’model’ logic (data processing, calculations) remains distinct from presentation ’view’ logic (display structure). This separation improves maintainability, reduces security vulnerabilities, and enables parallel development by allowing designers to modify presentation independently from programmers updating business logic [10].

Effective template systems must satisfy three key design criteria. First, they should enforce separation of concerns, ensuring business logic remains in the model rather than templates. Second, they must provide clear syntactic boundaries that distinguish presentation structure from application data, especially for their target language. Third, they must balance computational power appropriately.

This balance is critical. Parr (2004) demonstrates that template engines with greater

computational power (those allowing conditionals, arithmetic operations, and method calls) enable logic duplication and violate model-view separation when developers embed business rules directly in templates under deadline pressure [10]. The security implications are severe: recent analysis of 34 template engines found that only 29% provide security features, with most vulnerabilities stemming from excessive computational power in templates [11]. However, overly restrictive systems present their own challenges, requiring substantial data preprocessing before rendering and producing view models with excessive getter methods [3, 4]. The ideal template system restricts computational power to only what is necessary for presentation while avoiding unnecessary complexity in the model layer.

3.3. Alternative Templating Systems

Modern HTML Templating Engines:

- Handlebars extends Mustache by adding helpers, block expressions, and partials while maintaining logic-less philosophy, compiling templates into JavaScript functions for faster execution [12, 13]. However
- Nunjucks, inspired by Python’s Jinja2, provides block inheritance, autoescaping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining readable syntax [14]. It offers template inheritance with block overriding and automatic HTML escaping for XSS prevention.

Logic-Friendly Templating Languages:

- EJS (Embedded JavaScript) allows developers to write plain JavaScript directly in templates using special delimiters, with `<% %>` for code execution and `<%= %>` for output, eliminating the need to learn new syntax [15]. It caches intermediate functions for fast execution and provides full JavaScript expressiveness with server-side rendering support. However, EJS provides no security boundaries. Templates have direct access to the JavaScript runtime, making it unsuitable for scenarios involving untrusted template sources.
- Jinja2 is Python’s fast and extensible templating engine that supports template inheritance, macros, HTML autoescaping, and sandboxed environments for safely rendering untrusted templates [4, 16]. It offers more flexibility than Django’s templating system and features a powerful extension system with custom filters and tests. However, the sandbox has documented bypass vulnerabilities and security issues: attackers can exploit Python’s introspection mechanisms (`__class__`, `__subclasses__`) to escape restrictions, with recent exploits (CVE-2024-22195, CVE-2024-27516) demonstrating remote code execution despite sandbox protections [11].

Language-Integrated Templating:

- React/JSX integrates markup directly into JavaScript as first-class code through components, where JSX expressions compile to regular JavaScript function calls that can be used in conditionals, loops, and variable assignments [17]. It provides a component-based architecture with full JavaScript expressiveness and seamless TypeScript integration for type safety.
- Svelte shifts template processing from browser runtime to compile-time, generating highly optimized vanilla JavaScript without framework overhead, producing smaller bundles and better performance than runtime frameworks [18]. It eliminates runtime overhead through compile-time processing and provides a reactive programming model with minimal boilerplate code.

System	Computational Power	Preprocessing Burden	Security Model	Key Limitation
Mustache	Substitution only	High (all logic pre-computed)	No sandbox needed	Bloated view models
Handlebars	Limited helpers	Medium (helpers reduce burden)	Helper whitelisting	Helper proliferation
Nunjucks	Context-sensitive	Low (templates handle logic)	Sandboxing	Separation violations
Jinja2	Turing-complete	Minimal (full Python access)	Sandbox (by-passable)	SSTI vulnerabilities
EJS	Turing-complete	Minimal (full JavaScript)	None	Direct code injection
React/JSX	Turing-complete	Minimal (native JS)	Type checking only	Not logic-less
Svelte	Compile-time	Low (compiler handles logic)	Static analysis	Requires build step

Table 3.1: Template System Comparison (Author’s analysis applying Parr’s framework [10] and Pisu’s findings [11])

These alternatives reveal an inverse relationship: strict separation enforcement correlates with high preprocessing burden (Mustache, Handlebars), while unrestricted computational power correlates with low preprocessing but documented security vulnerabilities (Jinja2, EJS).

3.4. Functional Languages for Template Processing

Functional programming languages bring distinctive characteristics to template systems: strong static typing catching errors at compile-time, pattern matching for data structure manipulation, and algebraic datatypes for representing abstract syntax trees [7, 9]. Standard ML, OCaml, and Haskell share ML ancestry but differ in design philosophy.

Standard ML emphasizes formal rigor with complete formal definition [19, 20], strict evaluation making behavior predictable, and pattern matching naturally representing hierarchical data structures. However, its limited library ecosystem requires custom JSON parsing implementations [7]. OCaml prioritizes industrial pragmatism with aggressive compiler optimizations and richer standard library via OPAM [9]; Jingoo provides Jinja2-compatible templating for OCaml applications [21] but inherits Jinja2’s security model. Haskell’s lazy evaluation and advanced type system enable sophisticated solutions: Hamlet uses Template Haskell metaprogramming for type-checked template compilation [22], while Blaze-html represents HTML as composable functions for maximum type safety [23].

This project selects Standard ML for preprocessing automation rather than template engine implementation. Pattern matching naturally expresses transformations like “add `last: true` to final list element,” while exhaustive checking ensures all preprocessing cases are handled. The type system provides compile-time guarantees that required preprocessing fields (boolean flags, separators, format variations) are complete—catching omissions before runtime. While Standard ML’s limited ecosystem presents challenges, these tradeoffs accept implementation burden for verification guarantees.

3.5. Takeaway

These alternatives reveal distinct philosophical positions on the security-usability spectrum. No existing system successfully combines strict separation enforcement with low preprocessing burden: Mustache prioritises security at the cost of extensive manual data preparation [3, 4], while systems like Jinja2 reduce preprocessing but suffer documented security vulnerabilities [11]. While preprocessing requirements are well-documented as complaints, no work formalizes these patterns or provides type-safe automation. Addressing this gap, this project investigates whether Standard ML’s type system can automate preprocessing transformations while preserving logic-less templates’ security advantages.

4. Aims and Objectives

4.1. Proposal

This project investigates a Mustache template engine in Standard ML, by exploring two processing strategies: (1) preprocessing, where Standard ML transforms JSON data before template application, and (2) postprocessing, where Standard ML manipulates Mustache-generated output in Python [3]. The reason why Python was chosen as the target language is due to wide-scale and rising use, particularly with so many students learning machine learning. Python is especially well-suited for machine learning and artificial intelligence and is particularly approachable due to its easy syntax [24]. Both approaches leverage Standard ML’s strong static type system and pattern matching to address the data massaging challenges of logic-less templates. The implementation parses Mustache syntax, processes data contexts, and generates output while maintaining type safety throughout the pipeline. Beyond creating a functional tool, the project evaluates Standard ML’s suitability for template engine implementation in both preprocessing and postprocessing roles, examining advantages (type safety, pattern matching, parsing) and limitations (library ecosystem, tooling) encountered during development.

4.2. Research Questions

Three research questions guide this investigation:

1. **Formalization:** What preprocessing patterns (boolean flag generation, separator logic, format variations, computed display values) can be formalized as algebraic datatypes?
2. **Automation:** Can Standard ML’s pattern matching automate these transformations, reducing manual preprocessing burden while maintaining type safety?
3. **Verification:** Does compile-time verification catch preprocessing errors (missing flags, incomplete transformations) that would otherwise manifest at runtime?

4.3. Technical Approach

The project explores two processing strategies: **preprocessing**, where Standard ML transforms raw JSON data into Mustache-ready structures by automatically adding required fields (boolean flags, separator markers, format variations), and **postprocessing**, where Standard ML validates Mustache-generated output for correctness. Both approaches provide formal verification of template processing pipelines.

Beyond implementation, the project critically evaluates whether Standard ML's type safety advantages justify its limited ecosystem, examining the practical tradeoffs of applying formal methods to template preprocessing automation.

4.4. Learning Objectives

My learning objectives for this project include...

1. **Functional Programming:** Develop practical proficiency in Standard ML, including pattern matching, algebraic datatypes, and functional programming paradigms
2. **Testing and Debugging:** Gain experience with software testing practices and debugging in a functional programming context
3. **Research Skills:** Conduct systematic literature review of templating engine alternatives and critically evaluate technical tradeoffs
4. **Academic Writing:** Improve scholarly writing practices, including maintaining objective tone and structuring technical arguments
5. **Project Management:** Navigate academic milestones (interim report December 2024, final dissertation May 2025) while balancing thorough understanding with deadline constraints
6. **Implementation Experience:** Develop practical software development skills, including working with limited library ecosystems and parser implementation

5. Project Management

5.1. Project Overview and Timeline

The project commenced in the first week of October 2025 and runs through to final dissertation submission in May 2026. This interim report covers the initial research phase (October-December 2025), documenting progress to date and establishing a detailed plan for the implementation phase (January-May 2026). The overall project structure is visualized in Figure A.1 (Appendix A.1).

5.2. Key Milestones

- **Interim Report:** Monday 15th December 2025
- **Implementation Completion:** End of March 2026
- **Final Dissertation:** Mid-May 2026

5.3. Planned Work Schedule (October - December 2025)

5.3.1. Weeks 1-3 (Early October): Exploration and Topic Selection

Goals:

- Explore Standard ML fundamentals to determine dissertation focus
- Complete introductory exercises from Gilmore tutorial
- Experiment with basic SML concepts and functional programming patterns
- Identify project direction within functional programming domain

5.3.2. Weeks 4-6 (Mid-Late October): Topic Refinement

Goals:

- Continue Standard ML learning through Gilmore tutorial
- Research templating engines and functional language applications
- Finalize project direction: Mustache templating with Standard ML
- Submit aims and objectives
- Start working on first interim report draft to supervisor

5.3.3. Weeks 7-9 (November): Literature Review

Goals:

- Complete comprehensive review of templating approaches (Mustache, Nunjucks, Handlebars, EJS, Jinja2, Liquid, React/JSX, Svelte)
- Investigate Mustache's limitations and preprocessing challenges
- Identify research gap in type-safe template preprocessing
- Study Standard ML type systems and pattern matching capabilities
- Refine research questions based on literature findings

5.3.4. Weeks 10-11 (Early December): Interim Report Finalisation

Goals:

- Synthesize research findings
- Document technical approach and methodology
- Complete all interim report sections
- Develop detailed forward plan for implementation phase
- Submit the interim report

5.4. Forward Plan (January - May 2026)

The implementation and writing phases span January to May 2026, organized into six focused periods aligned with the high-level timeline in Figure A.1. Each phase builds upon previous work with clear deliverables and success criteria detailed in Section 5.5.

5.4.1. Weeks 12-13 (Christmas Break): SML Implementation Foundations

Goals:

- Work through simple Standard ML tutorials and exercises set by supervisor
- Build confidence with basic SML syntax and functional concepts
- Set up development environment for implementation work
- Review Mustache specification in detail

5.4.2. Weeks 14-17 (Late January - February): Pattern Matching Deep Dive

Goals:

- Be confident in Standard ML pattern matching techniques
- Study algebraic datatypes and their applications
- Practice with increasingly complex pattern matching examples
- Explore how pattern matching applies to template parsing
- Complete 3-4 weeks of focused exercises, which utilise specific SML properties (e.g., strong parsing abilities)

5.4.3. Weeks 18-19 (Late February - Early March): Mustache Implementation

Goals:

- Implement Mustache parser using SML pattern matching/preprocessing
- Develop preprocessing transformation logic
- Create data structures for template representation
- Build core functionality for boolean flags and separator handling

5.4.4. Weeks 20-22 (March): Testing and Refinement

Goals:

- Develop comprehensive test suite
- Test preprocessing transformations with various input patterns
- Debug and refine implementation
- Validate type safety guarantees
- Document edge cases and limitations

5.4.5. Weeks 23-26 (April - Early May): Dissertation Writing

Goals:

- Write implementation and evaluation chapters
- Analyze results and lessons learned
- Complete critical evaluation of SML's suitability
- Revise and polish all dissertation sections
- Final proofreading and submission preparation

5.4.6. Week 27 (Mid-May): Final Submission

Goals:

- Final dissertation review
- Submit completed dissertation

5.5. Deliverables and Success Criteria

Each project phase produces specific, measurable outputs that demonstrate progress and provide checkpoints for quality assurance:

5.6. Dependencies and Critical Path

The project follows a sequential dependency structure illustrated in Figure A.1. Understanding these dependencies is crucial for risk management and timeline adherence:

Phase	Deliverable	Success Criteria
Research Phase (Weeks 1-11)	Interim report documenting literature review and forward plan	Comprehensive templating engine comparison; clear research questions; detailed implementation plan
SML Foundations (Weeks 12-13)	Completed tutorial exercises; development environment setup	Ability to write basic SML programs; working compiler installation
Pattern Matching (Weeks 14-17)	Pattern matching exercises; parser design document	Confidence with algebraic datatypes; clear architecture for Mustache AST representation
Implementation (Weeks 18-19)	Working Mustache parser with preprocessing	Parses valid Mustache syntax; generates correct output; handles boolean flags and separators
Testing (Weeks 20-22)	Test suite with documented results	80%+ code coverage; edge cases documented; type safety validation complete
Dissertation (Weeks 23-27)	Complete dissertation meeting institutional requirements	All chapters complete; honest evaluation of approach; proper citations; professional formatting

Table 5.1: Phase Deliverables and Success Criteria

5.6.1. Sequential Dependencies

- **Weeks 1-11 → Weeks 14-17:** Literature review and SML learning inform implementation design decisions
- **Weeks 14-17 → Weeks 18-19:** Pattern matching mastery is prerequisite for parser implementation
- **Weeks 18-19 → Weeks 20-22:** Parser completion gates testing phase
- **Weeks 20-22 → Weeks 23-26:** Testing results required before evaluation chapter can be written

The critical path runs through pattern matching learning (Weeks 14-17) and implementation (Weeks 18-19), with 1 week buffer in testing phase.

5.7. Risk Management and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Pattern matching learning curve steeper than expected	Medium	Implementation delays (1-2 weeks)	Allocate buffer time, seek supervisor guidance, utilize online SML resources and communities
CM-Lex/CM-Yacc integration difficulties	Medium	Parser implementation delays	Research tools early, have hand-written parser as fall-back approach
Limited SML debugging tools	High	Extended debugging time	Write extensive unit tests, use print debugging, document issues for dissertation
Implementation doesn't meet expectations	Medium	Requires honest critical analysis	Frame as learning experience, document what worked/didn't work, valuable for research contribution
Time pressure in final month	Low	Rushed dissertation writing	Start writing early, document progress continuously, maintain weekly writing targets

Table 5.2: Implementation Phase Risk Assessment

6. Current Status of Work

6.1. Overview

This chapter documents substantial progress achieved during the research phase (October-December 2025), demonstrating completion of all foundational objectives. Literature review, technical foundation, and research justification are now complete.

6.2. Progress by Phase

6.2.1. Weeks 1-3 (Early October): Foundation

Completed introductory Gilmore tutorial exercises, establishing basic SML competency. Early progress was slower due to unfamiliarity with signatures, primary/secondary computation, and type systems. Met with Gergely to address conceptual gaps, building coherent understanding of SML’s design philosophy.

Achievements: Completed Gilmore tutorial introduction; identified knowledge gaps; established supervisor communication pattern; explored dissertation directions.

6.2.2. Weeks 4-6 (Mid-Late October): Topic Refinement

Progressed through Gilmore chapters 9-10 (signatures, modules). Developed date-handling code with supervisor-approved hardcoding. Produced first complete interim report draft. Finalized project direction: type-safe Mustache preprocessing in Standard ML.

Achievements: Completed chapters 9-10; wrote functional SML code; produced first report draft; submitted aims and objectives.

6.2.3. Weeks 7-9 (November): Literature Deep Dive

Conducted detailed analysis of 7+ templating engines (Nunjucks, Handlebars, EJS, Jinja2, Liquid, React/JSX, Svelte), identifying Mustache’s preprocessing challenges and research gap. Iteratively refined report based on supervisor feedback. Gergely’s advice to “read on a different day” proved valuable—fresh perspective consistently revealed clarity improvements.

Achievements: Completed comprehensive templating comparison; identified research gap; refined research questions; incorporated supervisor feedback; shifted from descriptive to analytical framing.

6.2.4. Weeks 10-11 (Early December): Finalization

Finalized interim report through final revisions, ensuring coherent arguments with appropriate citations. Completed forward plan with realistic timelines informed by actual learning challenges.

Achievements: Completed interim report for submission; established implementation roadmap; documented risk assessment.

6.3. Key Accomplishments

Category	Completed Deliverables
Technical Foundation	Gilmore tutorial completed; working SML code examples; understanding of algebraic datatypes, pattern matching, type safety
Research	Comprehensive comparison of 7+ templating engines; research gap identified and justified; three focused research questions established
Documentation	Complete interim report; project management framework; Harvard-style referencing implemented

Table 6.1: Research Phase Deliverables

6.4. Self-Review

Coming from Biomedical Engineering, the computer science knowledge gap was significant. Concepts like signatures, primary/secondary computation, and algebraic datatypes required substantial background reading. Early progress was slower than anticipated, necessitating additional foundational time.

However, this careful groundwork proved essential. Time invested in understanding SML’s type system will directly support implementation. Deep engagement with templating alternatives strengthened research justification by revealing genuine gaps. Supervisor feedback developed ability to identify unclear writing; revisiting work with fresh perspective improved clarity and analytical depth.

Research phase successfully established implementation prerequisites: Basic SML competency achieved; research clarity established; realistic planning completed; support structures working; problem space mapped.

All major objectives achieved:

- Identified specific research direction
- Completed comprehensive literature review
- Developed foundational SML competency
- Produced complete interim report
- Established realistic forward plan

Project Status: On track for January 2026 implementation phase. Research base is solid, technical direction is clear, and realistic planning accounts for continued learning during implementation.

A. Project Timeline

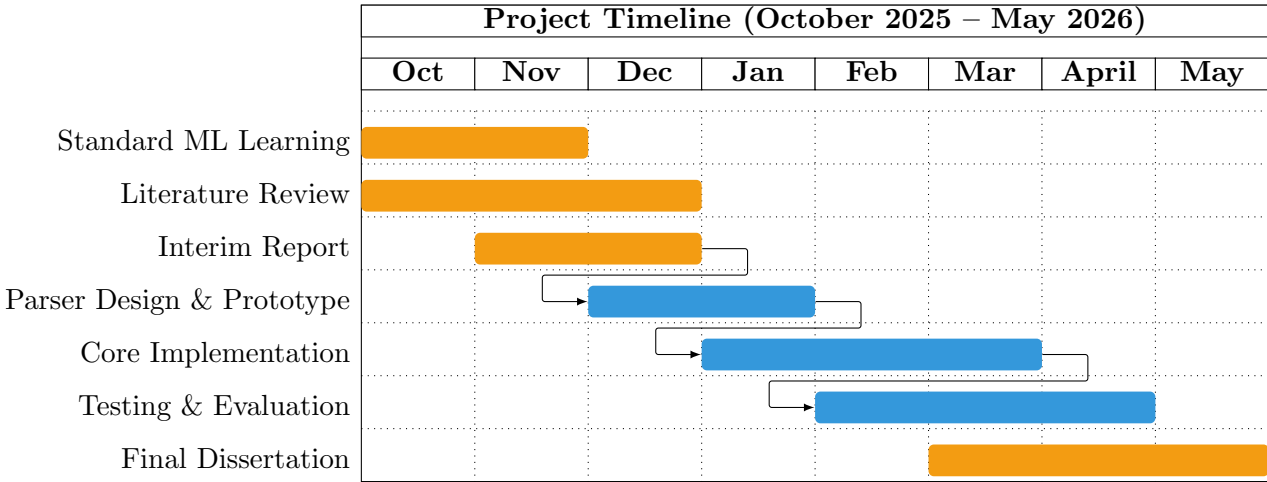


Figure A.1: High-level project Gantt chart showing major phases from October 2025 to May 2026. Orange bars indicate research and writing phases; blue bars indicate implementation work. Detailed weekly breakdown provided in Sections 2.3 and 2.4. The critical path runs through pattern matching learning (Feb) and core implementation (Feb-Mar).

Bibliography

- [1] T. J. Parr, “Enforcing strict model-view separation in template engines,” in *Proceedings of the 13th International Conference on World Wide Web*, ser. WWW ’04. New York, NY, USA: ACM, May 2004, pp. 224–233.
- [2] “Mustache - logic-less templates,” Official Website. [Online]. Available: <https://mustache.github.io/>
- [3] S. Pottavathini. (2012, Oct.) The case against logic-less templates. eBay Inc. [Online]. Available: <https://innovation.ebayinc.com/stories/the-case-against-logic-less-templates/>
- [4] A. Boronine. (2012, Sep.) Cult of logic-less templates. Alexei Boronine’s Blog. [Online]. Available: <https://www.boronine.com/2012/09/07/Cult-Of-Logic-less-Templates/>
- [5] “Mustache (template system),” Wikipedia, accessed: 2025-11-21. [Online]. Available: [https://en.wikipedia.org/wiki/Mustache_\(template_system\)](https://en.wikipedia.org/wiki/Mustache_(template_system))
- [6] A. W. Appel and D. B. MacQueen, “A standard ml compiler,” *Technical Report, Department of Computer Science, Princeton University*, 1997, accessed: 2025-11-21. [Online]. Available: <https://www.cs.princeton.edu/~appel/papers/97.pdf>
- [7] Y. U. Flint, “The case for ml,” n.d., accessed: 2025-11-21. [Online]. Available: <https://flint.cs.yale.edu/cs421/case-for-ml.html>
- [8] *Mustache Manual*, Mustache. [Online]. Available: <https://mustache.github.io/mustache.5.html>
- [9] “Ml (programming language),” Wikipedia, accessed: 2025-11-21. [Online]. Available: [https://en.wikipedia.org/wiki/ML_\(programming_language\)](https://en.wikipedia.org/wiki/ML_(programming_language))
- [10] T. J. Parr, “Enforcing strict model-view separation in template engines,” in *Proceedings of the 13th International Conference on World Wide Web*, ser. WWW ’04. New York, NY, USA: ACM, 2004, pp. 224–233.

- [11] L. Pisu, D. Maiorca, and G. Giacinto, “A survey of the overlooked dangers of template engines,” *arXiv preprint arXiv:2405.01118*, May 2024. [Online]. Available: <https://arxiv.org/abs/2405.01118>
- [12] J-Gallo. (2016, Feb.) Mustache vs Handlebars. [Online]. Available: <https://medium.com/@JuaniGallo/mustache-vs-handlebars-42e531eca252>
- [13] H. Team, “Handlebars,” Official Website, accessed: 2025-11-21. [Online]. Available: <https://handlebarsjs.com/>
- [14] Mozilla, “Nunjucks: A powerful templating engine with inheritance,” GitHub, accessed: 2025-11-21. [Online]. Available: <https://mozilla.github.io/nunjucks/>
- [15] E. Team, “Ejs – embedded javascript templates,” Official Website, accessed: 2025-11-21. [Online]. Available: <https://ejs.co/>
- [16] Pallets, “Jinja,” Official Website, accessed: 2025-11-21. [Online]. Available: <https://palletsprojects.com/p/jinja/>
- [17] R. Team, “Writing markup with jsx,” React Documentation, accessed: 2025-11-21. [Online]. Available: <https://react.dev/learn/writing-markup-with-jsx>
- [18] S. Team, “Svelte documentation,” Official Website, accessed: 2025-11-21. [Online]. Available: <https://svelte.dev/docs>
- [19] “Standard ml,” Wikipedia, accessed: 2025-11-21. [Online]. Available: https://en.wikipedia.org/wiki/Standard_ML
- [20] D. MacQueen, R. Harper, and J. Reppy, “The history of standard ml,” *Proceedings of the ACM on Programming Languages*, vol. 4, no. HOPL, 2020. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3386336>
- [21] T. Bunko, “Jingoo: Template engine for ocaml,” GitHub. [Online]. Available: <https://github.com/tategakibunko/jingoo>
- [22] “Shakespearean templates,” Yesod Book. [Online]. Available: <https://www.yesodweb.com/book/shakespearean-templates>
- [23] J. Van Der Jeugt, “Blazehtml: A blazingly fast html combinator library,” Personal Website. [Online]. Available: <https://jaspervdj.be/blaze/>
- [24] M. Moore and A. Gureiev. (2024, January) 10 compelling reasons to choose python for your next project. Accessed: 2025-11-26. [Online]. Available: <https://shakuro.com/blog/ten-reasons-to-choose-python>